End2Race: Efficient End-to-End Imitation Learning for Real-Time F1Tenth Racing

Zhijie Qiao[1,†], Haowei Li[1,†], Zhong Cao[1], Henry X. Liu[1,2,*]

***Abstract*— F1Tenth is a widely adopted reduced-scale platform for developing and testing autonomous racing algorithms, hosting annual competitions worldwide. With high operating speeds, dynamic environments, and head-to-head interactions, autonomous racing requires algorithms that diverge from those in classical autonomous driving. Training such algorithms is particularly challenging: the need for rapid decision-making at high speeds severely limits model capacity. To address this, we propose End2Race, a novel end-to-end imitation learning algorithm designed for head-to-head autonomous racing. End2Race leverages a Gated Recurrent Unit (GRU) architecture to capture continuous temporal dependencies, enabling both short-term responsiveness and long-term strategic planning. We also adopt a sigmoid-based normalization function that transforms raw LiDAR scans into spatial pressure tokens, facilitating effective model training and convergence. The algorithm is extremely efficient, achieving an inference time of less than 0.5 milliseconds on a consumer-class GPU. Experiments in the F1Tenth simulator demonstrate that End2Race achieves a 94.2% safety rate across 2,400 overtaking scenarios, each with an 8-second time limit, and successfully completes overtakes in 59.2% of cases. This surpasses previous methods and establishes ours as a leading solution for the F1Tenth racing testbed. Code is available at https://github.com/michigan-traffic-lab/End2Race.**

I. Introduction

Autonomous racing has emerged as a compelling testbed for advancing autonomous vehicle technologies. It combines perception, planning, and control under high-speed, interactive, and computationally constrained conditions. Enhancing algorithmic capabilities in this domain is therefore relevant to the broader development of autonomous driving technologies. F1Tenth [1], a 1/10-scale vehicular platform, provides an accessible testbed for autonomous racing education and research. Equipped with a 2D LiDAR [2] or RGB-D camera [3] for perception, the vehicle can reach speeds of up to 17.9 m/s, equivalent to approximately 400 mph at full scale [4]. Each year, competitions are held at venues worldwide and attract broad participation from the community.

Existing approaches for autonomous racing can be broadly grouped into map-based, reactive, and learning-based. Of these, map-based methods are the most commonly used in F1Tenth competitions. They rely on a pre-constructed map to follow a globally optimized raceline while adapting to local conditions. In the mapping phase, the vehicle is manually controlled along the track to construct the map using SLAM [5]. Raceline optimization [6] then uses the mapped geometry to derive a global reference path. During racing, particle-filter localization [7] combines odometry and LiDAR measurements to estimate the vehicle pose within the map. Using this estimate, a lattice planner [8] generates candidate trajectories that progress along the raceline while satisfying dynamic and collision constraints. Once a trajectory is selected, its velocity profile provides the speed command, while a path-tracking controller, such as a Pure Pursuit controller [9] or model predictive control (MPC) [10], generates the steering command. This design enables the vehicle to anticipate upcoming curves and consistently track the reference path. Yet map-based methods have two principal limitations. First, they require a preprocessing stage and cannot operate on previously unseen tracks. Second, on an F1Tenth vehicle's onboard computer [4], localization [11] and planning [12] together incur a systematic delay of approximately 30 milliseconds. With additional sensor latency, the vehicle can travel up to a full vehicle length before updated control commands take effect. This delay is non-critical under most operating conditions but becomes consequential when a high-speed straight segment is directly followed by a sharp curve, leaving the vehicle susceptible to collisions with the track boundaries. Such collisions are widely observed in F1Tenth competitions, including during semifinal and final rounds.

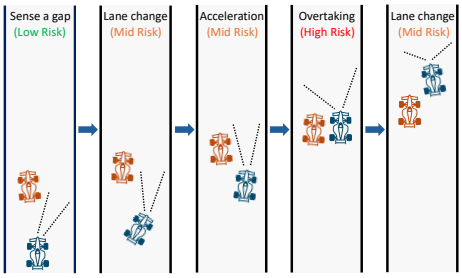

Fig. 1. Simple overtake scenario illustration.

Reactive methods provide a simpler, rule-based alternative by acting directly on current sensor measurements. Their map-free design avoids localization and planning, reducing computational cost while allowing operation on unknown tracks. The Follow-the-Gap algorithm [13], for example, identifies the largest free region and steers toward its midpoint. However, these methods operate instantaneously, as each decision is based on an isolated observation with limited temporal context, leading to planning inconsistencies and

This research was partially funded by the DARPA TIAMAT Challenge (HR0011-24-9-0429).

[1]Z. Qiao, H. Li, Z. Cao, and H. X. Liu are with the Department of Civil and Environmental Engineering, University of Michigan, Ann Arbor, MI 48109, USA.

[2]H. X. Liu is also with University of Michigan Transportation Research Institute, Ann Arbor, MI 48109, USA.

†These authors contributed equally to this work.

*Corresponding author: Henry X. Liu (henryliu@umich.edu).

control instability [14].

Learning-based methods have also been widely proposed for F1Tenth. Early work focused primarily on single-vehicle navigation [15], [16], whereas more recent studies have shifted toward head-to-head competition. However, existing approaches either use only static obstacles during training before being tested directly against dynamic opponents [17], rely on a small set of human demonstrations and evaluation scenarios [18], or operate at low racing speeds [19].

In high-speed head-to-head competition, overtaking is a key determinant of the race outcome and presents an intrinsically challenging problem. Figure 1 illustrates a simplified five-stage overtaking process. The ego vehicle first identifies a suitable opening and then executes a sharp lane change to align with it while accelerating rapidly to pass, increasing the risk of losing control. Throughout the pass, the vehicles remain side by side for an extended period, with little clearance between them and from the track boundaries. Finally, the ego vehicle must quickly return to the track center to avoid a boundary collision. Successful overtaking therefore depends on continuous decision-making, precise maneuver execution, rapid adaptation to the opponent's motion, and minimal sensing-to-control latency. This requires both sufficient model capacity to represent complex conditions and low inference latency, a combination not yet achieved by existing approaches.

To address these limitations, we propose **End2Race**, an end-to-end learning framework for head-to-head autonomous racing that enables systematic training and evaluation of overtaking policies at scale. End2Race takes a 360-degree 2D LiDAR scan and the current vehicle state as inputs and directly produces control actions. To encode temporal dependencies efficiently, we adopt a Gated Recurrent Unit (GRU) network [20] that maintains a hidden state summarizing the observation history. On a physical F1Tenth vehicle's onboard computer, the network produces control actions with an inference time below 1 millisecond. Training follows a two-stage pipeline in which the policy is first trained through behavioral cloning [21] using demonstrations generated by a map-based expert and then fine-tuned through reinforcement learning using PPO [22]. Across both seen and unseen tracks, End2Race demonstrates strong single-vehicle racing performance as well as competitive overtaking skills.

The main contributions of this work are as follows:

1) We introduce End2Race, an end-to-end learning framework for head-to-head autonomous racing that enables systematic training and evaluation of overtaking scenarios at scale.
2) We design a computationally efficient recurrent policy network with a sensing-to-control latency below 1 ms, improving safety and supporting real-time operation at high racing speeds.
3) Through a two-stage training pipeline combining behavioral cloning with reinforcement learning fine-tuning, End2Race generalizes across unseen tracks and demonstrates strong single-vehicle and head-to-head racing performance.

II. Related Work

A. Imitation Learning

Imitation learning (IL) trains a driving policy from expert demonstrations. Sun et al. [15] generated action labels using a Pure Pursuit controller, then evaluated behavioral cloning [21], DAgger [23], HG-DAgger [24], and expert intervention learning [25]. All four policies completed the training track but failed to complete an unseen track. Paluch et al. [26] trained a feedforward policy to imitate nonlinear model predictive control along a precomputed raceline. The learned policy completed repeated laps faster than a Pure Pursuit controller. TinyLidarNet [17] trained a compact convolutional policy from human demonstrations involving only static obstacles and then evaluated it directly against dynamic opponents. Zhang and Loidl [18] trained dense and recurrent policies from human overtaking demonstrations, but their study used limited training data and evaluated the policies using only a small set of scenarios.

B. Model-Free Reinforcement Learning

Model-free reinforcement learning (RL) learns a control policy from observations to maximize cumulative reward. Hamilton et al. [27] compared DDPG [28], SAC [29], and behavioral cloning [21], finding that the RL policies performed better in racing but incurred more collisions. Steiner et al. [19] trained TD3 [30] for head-to-head racing and showed that opponent-inclusive training improved overtaking success against multiple controllers. However, their ego vehicle was limited to 2 meters per second, far below practical racing speeds. High-speed racing constitutes a different regime in which shorter reaction times and stronger dynamic effects demand greater closed-loop stability, real-time responsiveness, and maneuver precision.

C. Model-Based Reinforcement Learning

Model-based RL learns how the vehicle and environment respond to actions, then uses this predictive model to evaluate possible future outcomes. Dreamer [31], for example, learns a recurrent latent dynamics model and trains the policy using trajectories imagined in that latent state. Brunnbauer et al. [16] applied Dreamer to autonomous racing and outperformed model-free agents in task completion and generalization. Meanwhile, performance on unseen tracks varied with the policy inputs.

D. Hybrid Learning Methods

Hybrid methods retain a structured planner or controller within the learned pipeline. Dwivedi et al. [32] incorporated reference trajectories from an external planner into an RL controller, allowing the policy to learn continuous control while retaining high-level planning guidance. Bhargav et al. [33] estimated overtaking success across lateral positions for each track segment and used these estimates to select normal, left, or right planner modes. Trumpp et al. [34] combined residual policy learning with an artificial-potential-field planner for mapless multi-agent racing.

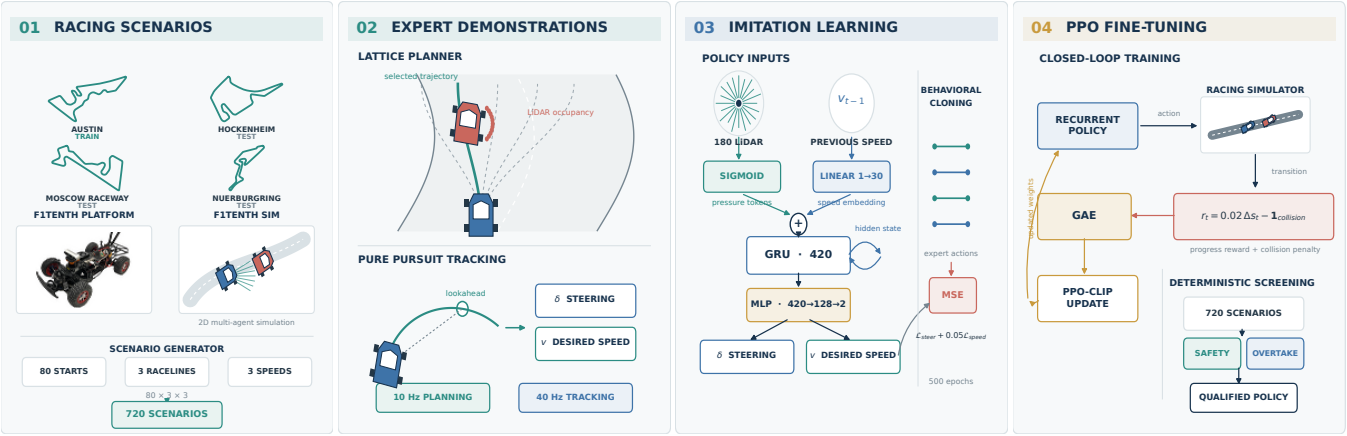

Fig. 2. System overview. The F1TENTH vehicle photograph in panel 1 is adapted from the official F1TENTH documentation under CC BY-NC-SA 4.0.

III. Methodology

We develop and evaluate End2Race using the F1TENTH Gym simulator [1], which provides high-fidelity vehicle dynamics and accurate 2D LiDAR simulation. Four racing tracks are selected, each reflecting the layout of a real-world Formula 1 circuit: Austin, Hockenheim, MoscowRaceway, and Nuerburgring (Fig. 2(a)). The Austin track is used for training, while the remaining tracks are reserved for evaluation. Within this environment, an automated workflow generates overtaking scenarios at scale across diverse track geometries, raceline configurations, and racing speeds, with each scenario pairing an ego vehicle with a leading opponent. A map-based expert controls the ego vehicle and generates demonstrations for policy training (Fig. 2(b)). We first train a Gated Recurrent Unit (GRU) network through behavioral cloning (Fig. 2(c)), and then fine-tune the resulting policy through reinforcement learning with PPO (Fig. 2(d)). The following sections describe each stage in detail.

A. Overtaking Scenario Setup

Each overtaking scenario pairs an ego vehicle with an opponent initially positioned ahead of it and has a duration of 8 seconds. Vehicle placement is expressed in Frenet coordinates [35], in which the longitudinal direction measures progress around the circuit and the lateral direction specifies position across the track width. We specify each scenario using four parameters, $(s_0,\, d_0,\, r_{\text{opp}},\, \alpha_{\text{opp}})$, as detailed below.

1) Ego Starting Position (s_0): For each track, we define N_s ego starting positions equally spaced in the longitudinal direction, with $N_s = 80$ fixed across all tracks. For example, on the 420 m Austin track, adjacent starting positions are separated by approximately 5.2 m.

2) Initial Distance (d_0): The initial distance d_0 specifies how far the opponent is positioned longitudinally ahead of the ego vehicle at the start of a scenario. A short distance provides insufficient starting space for the ego vehicle to maneuver, while a long distance requires the ego vehicle to spend more time catching the opponent, leaving less time for interaction. We therefore set $d_0 = 3$ m to balance these considerations.

3) Opponent Raceline (r_{opp}): Beyond their longitudinal placement, the relative lateral positions of the ego vehicle and opponent are critical to the overtaking geometry. We represent this configuration using racelines, which define the reference paths followed by the vehicles around the circuit. Using the trajectory-generation framework of Heilmeier et al. [6], we construct left, center, and right racelines at 35%, 50%, and 65% of the local track width, respectively. For Austin, an average track width of 1.64 m yields an approximately 0.25 m separation between the center and each side raceline. The ego vehicle starts on the center raceline but may temporarily deviate from it to overtake or take a faster line through a corner. The opponent, by contrast, follows the raceline specified by r_{opp}, with all three choices equally represented across the scenarios. An opponent on the center raceline leaves limited clearance on both sides and is therefore harder to pass, whereas one on a side raceline leaves wider passing space.

4) Opponent Speed Scale (α_{opp}): To facilitate effective overtaking behavior, the ego vehicle must have a speed advantage over the opponent. Here, we define a maximum vehicle speed of 7.5 m/s, equivalent to approximately 168 mph at full scale. This limit is based on empirical testing, which showed that higher speeds compromise control stability under the vehicle's dynamic constraints. The ego vehicle retains this limit, while the opponent's raceline speed profile is scaled by a factor of α_{opp}. We use $\alpha_{\text{opp}} \in \{0.4, 0.6, 0.8\}$, corresponding to maximum opponent reference speeds of $\{3.0, 4.5, 6.0\}$ m/s, respectively. Values below 0.4 make the opponent unrealistically slow for competitive racing, while values above 0.8 often prevent the ego vehicle from catching up to the opponent within the 8-second scenario.

Together, s_0 and d_0 determine the vehicles' initial longitudinal configuration, while r_{opp} and α_{opp} specify the opponent's path and speed profile. The opponent passively tracks its assigned raceline using a Pure Pursuit controller and does not engage in adversarial behavior, such as active blocking or abrupt acceleration [36]. With d_0 fixed, the 80 starting positions, three opponent racelines, and three speed scales yield 720 scenarios per track, each lasting 8 seconds.

B. Expert Demonstration

To generate demonstrations for the ego vehicle, we design a map-based expert that combines a lattice planner with a Pure Pursuit controller. The lattice planner, adapted from the Frenet Optimal Trajectory implementation in PythonRobotics [37], converts each LiDAR scan into obstacle points and generates candidate trajectories across a range of lateral positions and terminal speeds. It then discards candidates that violate the vehicle's dynamic constraints and, among the remaining candidates, selects the trajectory τ that minimizes the trajectory cost $\mathcal{J}(\tau)$:

$$\mathcal{J}(\tau) = \frac{1}{N}\sum_{i=1}^{N}\left(1 - \frac{v_i}{v_{\max}}\right) + \lambda_c \max_i \left[\max\left(0, 1 - \frac{d_i}{d_c}\right)\right]^2 \tag{1}$$

where

- $N = 6$ is the number of future trajectory points sampled at 0.1 s intervals over the 0.6 s planning horizon;
- v_i is the projected vehicle speed at point i;
- $v_{\max} = 7.5$ m/s is the speed limit for the ego vehicle;
- d_i is the distance from point i to the nearest obstacle;
- $d_c = 0.5$ m is the clearance threshold for applying the collision penalty;
- $\lambda_c = 5$ is the collision-avoidance penalty weight.

The first term favors trajectories with higher mean speeds, while the second penalizes the maximum clearance deficit along the trajectory. After selecting a trajectory, the Pure Pursuit controller identifies a lookahead point, computes the steering command from its lateral displacement relative to the ego vehicle, and uses the trajectory speed at that point as the desired speed.

C. Training Dataset

Using the procedure described above, we generate 720 eight-second overtaking scenarios on each track. The Austin track provides the training demonstrations, while the remaining tracks are reserved for evaluation (Fig. 2(a)). Training data are recorded at 40 Hz to reflect the onboard LiDAR-to-control update cycle (Fig. 2(b)). Each complete scenario contains 320 observation-action pairs, yielding up to 230,400 samples. Among the 720 Austin scenarios, 620 (86.1%) resulted in successful overtakes, 23 (3.2%) in collision-free car-following, and 77 (10.7%) in collisions.

The expert's failure mode stems primarily from the lattice planner's forward-only obstacle evaluation, which does not account for a moving opponent alongside or behind the ego vehicle. Consequently, the planner may return to the center lane before the ego vehicle has fully cleared the opponent. The number of failures increases as the speed difference decreases, from 4 cases at a speed scale of 0.4 to 15 at 0.6 and 58 at 0.8. Figure 3 illustrates a scenario in which this failure occurs, with the opponent following the right raceline at a speed scale of 0.8. During overtaking, the planned trajectory of the ego vehicle shifts toward the center lane while the ego and the opponent still overlap longitudinally, resulting in a rear sideswipe collision.

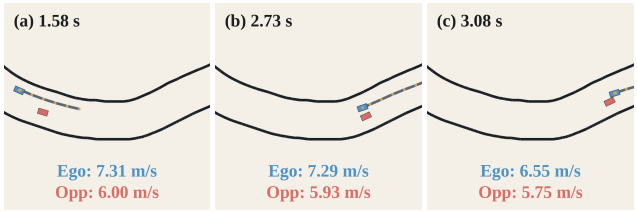

Fig. 3. Expert-planner failure during overtaking, shown in temporal order from (a) to (c). In this scenario, the opponent follows the right raceline with a speed scale of 0.8. Each frame shows the planned ego trajectory and both vehicle speeds; blue denotes the ego vehicle and red the opponent. (a) The ego approaches the opponent. (b) The vehicles travel side by side during overtaking. (c) The ego returns to the center lane prematurely, leading to a sideswipe collision.

For behavioral-cloning training (Sec. III-E.1), we exclude demonstrations from the 77 ego-collision scenarios and retain 643 collision-free sequences (89.3%), yielding 205,760 training samples. However, filtering alone does not provide the model with the experience needed to learn safe responses in such scenarios. Such failures can be mitigated by combining explicit opponent detection and motion prediction with interaction-aware planning as in [38]. The approach, however, requires considerable manual design and cost adjustment, incurs additional computation, and suffers from error propagation across modules. We instead address this failure mode through direct end-to-end reinforcement learning fine-tuning (Sec. III-E.2), keeping the model intentionally lightweight while enhancing its opponent-interaction capabilities.

D. Model Architecture

We adopt an end-to-end policy that maps sensory inputs and the current vehicle state directly to control outputs. At each timestep t, the policy receives a 360-degree 2D LiDAR scan comprising 180 beams at an angular resolution of 2 degrees. The current ego speed is also provided as input to maintain dynamic consistency across consecutive model predictions. The policy outputs the desired vehicle speed in meters per second and the steering angle in radians. To capture temporal dependencies in ego and opponent motion, we use a Gated Recurrent Unit (GRU), an architecture that uses reset and update gates to selectively retain relevant history and incorporate new information over time. In End2Race, the hidden state summarizes previous observations and provides temporal context for each control prediction.

1) LiDAR Representation: For LiDAR processing, the scan $\mathbf{z}_t$ is normalized elementwise into spatial pressure tokens using a sigmoid function $\sigma(\cdot)$, defined as

$$\sigma(x) = \left(-\frac{1}{1 + e^{-kx}} + 1\right) \cdot 2 \tag{2}$$

where x denotes a single LiDAR scan beam of $\mathbf{z}_t$, and the fixed coefficient $k = 0.3$ determines the effective sensing range. This normalization maps each raw scan value, $x \in [0, \infty)$, to a spatial pressure token within the range $[0, 1]$. The S-shaped curve of the sigmoid ensures that large scan distances x are mapped to values that asymptotically

approach zero, meaning distant obstacles have minimal influence on the network. As obstacles become closer, the token value increases rapidly and nonlinearly, making the system much more sensitive to nearby objects. At the same time, the maximum value is bounded at 1 as x approaches zero, ensuring stable normalization outputs. In this way, the LiDAR inputs are represented in a nonlinear, continuous, and differentiable manner that naturally aligns with human cognition. This facilitates more efficient training and enables the model to converge more effectively.

2) Ego-Speed Embedding: The most recently measured ego speed $v_{t-1} \in \mathbb{R}$ is projected through a learned linear layer into a 30-dimensional embedding $\psi(v_{t-1})$. During behavioral-cloning training, we prevent shortcut learning [39], where the model simply copies the ego-speed input to its output, by randomly replacing this embedding with a learnable mask:

$$\widetilde{\psi}(v_{t-1}) = \begin{cases} \psi(v_{t-1}), & \text{with probability } 1-p \\ \mathbf{e}_{\text{mask}}, & \text{with probability } p. \end{cases} \tag{3}$$

We set $p = 0.2$, encouraging the model to extract meaningful patterns from the LiDAR input rather than relying primarily on the ego speed. Masking is disabled during inference, when the projected ego speed is always used.

3) Recurrent Control Prediction: Concatenating the 180 spatial pressure tokens with the 30-dimensional ego-speed embedding produces a 210-dimensional GRU input. At each timestep, the GRU combines this input with the preceding hidden state:

$$\mathbf{h}_t = \mathrm{GRU}_\theta\left(\sigma(\mathbf{z}_t) \| \psi(v_{t-1}),\, \mathbf{h}_{t-1}\right), \tag{4}$$

where $\mathbf{h}_t \in \mathbb{R}^{420}$ is the updated hidden state, with twice the dimensionality of the GRU input. A two-layer MLP decoder head ($420 \to 128 \to 2$) then maps $\mathbf{h}_t$ to the steering-angle and desired-speed commands, $\mathbf{a}_t = [\delta_t, v_t^{\text{des}}]$.

E. Policy Training

1) Behavioral Cloning: To preserve temporal dependencies, each scenario input sequence is considered in its entirety, producing predictions at each timestep and aggregating losses across the full temporal horizon. The loss function is defined as a weighted sum of mean squared errors (MSE) for both the ego vehicle's speed and steering angle, balancing their relative scales and jointly guiding the model toward accurate control.

$$\mathcal{L} = 0.05\,\mathcal{L}_{\text{speed}} + \mathcal{L}_{\text{steer}} \tag{5}$$

2) Reinforcement Learning Fine-Tuning: TBD

IV. Experiments

This section provides implementation details and reports experimental results on the training track as well as three unseen test tracks. We first present the model's performance in the single-agent setting, showing detailed driving metrics. Next, we evaluate its behavior across 2,400 overtaking scenarios spanning all four tracks, analyzing outcomes under diverse conditions. Two comparative examples are shown in Fig. 4 and 5, illustrating a successful overtaking maneuver and safe car-following, respectively. Finally, we conduct stress tests with varying levels of sensor noise, which is essential for ensuring reliable operation in real-world autonomous racing applications.

A. Implementation Details

The model is trained for 500 epochs using the Adam optimizer, initialized with a learning rate of 0.001. To facilitate convergence, a learning rate scheduler monitors the training loss and automatically halves the rate whenever progress stalls. Training is conducted with mini-batches of size 16 and *completes in 10 minutes on a single NVIDIA 4080 GPU*, converging to a near-zero final loss.

B. Single-Agent Navigation

For each track, the ego vehicle is evaluated by performing ten consecutive laps, which aligns with standard practice in F1Tenth competitions. Table I summarizes the driving metrics. Across all tracks, the model successfully completes ten laps without collision, even on challenging sections featuring sharp turns and continuous sweeping curves. The mean speed remains high at about 7.34 meters per second, which corresponds to 165 miles per hour in full-scale racing. The low speed variance indicates consistent driving, with only moderate deceleration at sharper corners, while the low lap time variance further supports this claim. These results alone demonstrate the effectiveness of our model, achieving a level of performance not previously attained by imitation learning methods in the F1Tenth domain.

C. Head-to-Head Racing

Across all 2,400 overtaking scenarios conducted on four challenging tracks, each lasting 8 seconds, the model consistently achieves a high safety rate, exceeding 93.5% on every track and averaging 94.2%. The overtaking success rate is similarly strong, with an average of 59.2% across all tracks. In cases where overtaking cannot be performed safely, the model reliably switches to safe car-following behavior. Moreover, the model demonstrates consistent performance across all tracks, exhibiting negligible difference between the training and evaluation tracks, indicating that it acquires robust racing strategies that generalize beyond the training environment. This result is comparable to the map-based expert in terms of both safety and overtaking rate, representing the first imitation learning approach in the F1Tenth literature to reach this level of competency. In addition, our model does not require pre-mapping or fine-tuning, as is necessary for the map-based expert, and can be deployed directly on any unseen F1Tenth race track.

Relative to prior work [40], which reported an overtaking safety rate of 78.8%, our approach reduces collisions by a substantial margin of 72.6%. More importantly, while their evaluation was conducted on tracks with simple geometries, we deliberately selected tracks with sharp corners and narrow corridors to create more challenging overtaking scenarios.

Note that due to the time constraint in each scenario, overtaking opportunities are inherently limited. Extending

TABLE I
SINGLE-VEHICLE PERFORMANCE

	Mean Speed (m/s)	Speed Variance (m/s^2)	Mean LapTime (s)	LapTime Variance (s^2)	Laps Completed
Austin (Train)	6.93	0.78	60.44	1.19	10
Hockenheim	7.40	0.50	48.81	0.34	10
MoscowRaceway	7.21	0.54	44.71	0.11	10
Nuerburgring	7.80	0.39	57.56	0.21	10
Summary	*7.34*	*0.55*	*52.88*	*0.46*	*10*

TABLE II
HEAD-TO-HEAD RACING PERFORMANCE

	Car Following	Overtaking	Collision	Overtake Rate (%)	Safety Rate (%)
Austin (Train)	234	341	25	56.8	95.8
Hockenheim	233	330	37	55.0	93.8
MoscowRaceway	194	367	39	61.2	93.5
Nuerburgring	178	384	38	64.0	93.7
Summary	*210*	*355*	*35*	*59.2*	*94.2*

the duration (not adopted here to avoid unnecessarily longer training and testing) would substantially increase the overtaking rate of the model. In real F1Tenth competitions, overtakes are rare, and even a single successful maneuver across all 10 laps can provide a decisive advantage over an opponent.

D. Scenario Illustration

We illustrate an overtaking and a car-following scenario with continuous time frames (Fig. 4 and 5). The two scenarios feature nearly identical initial conditions, differing only in the leading vehicle's velocity, highlighting the contrastive behavior. In each case, the vehicles are navigating a continuous sweeping curve that consists of a left turn immediately followed by a right turn. The ego vehicle is shown in blue and the leading vehicle in red, with their trajectories depicted in the corresponding color.

In this setup, the leading vehicle follows the centerline of the track, leaving minimal gaps on either side, which makes overtaking especially challenging. Initially, the ego vehicle follows the leading vehicle (Fig. 4.1). As they approach the upcoming right turn, a brief opening appears, and the ego vehicle seizes the opportunity by steering sharply to the right and accelerating (Fig. 4.2 and 4.3). Upon entering the turn, the ego vehicle utilizes the inner lane and the shorter path to quickly overtake the leading vehicle. Immediately after passing, it reduces speed to avoid excessive velocity through the curve, which could result in loss of control (Fig. 4.4). After that, the ego vehicle resumes its pace (Fig. 4.5).

In the car-following scenario, the leading vehicle travels at a higher speed and maintains a larger distance ahead of the ego vehicle (Fig. 5.1). As in the overtaking example, the ego vehicle initially attempts to overtake by steering to the right (Fig. 5.2). However, midway through the maneuver, the gap closes and overtaking becomes infeasible. Instead of forcing an unsafe pass, the ego vehicle aborts the attempt and steers back behind the leading vehicle to resume following (Fig. 5.3 and 4). After both vehicles complete the turn, they accelerate back to cruising speed (Fig. 5.5).

E. Evaluation with Sensor Noise

In real-world autonomous racing, sensor noise remains a persistent and significant challenge. A common issue is the intermittent return of zeros from some LiDAR beams, caused by hardware malfunctions, environmental interference, or communication errors [41].

These anomalies can significantly degrade system performance. For example, in reactive methods such as Follow-the-Gap, the absence of even a single beam across a wide gap may entirely obstruct the passage, resulting in suboptimal path selection. In map-based methods like the lattice planner, missing measurements can cause localization errors and often lead the vehicle to collide with track boundaries. To the best of our knowledge, the impact of sensor noise has not been investigated in existing F1Tenth learning algorithms.

We assess the robustness of our model by randomly setting a proportion of LiDAR beams to zero at each timestep. Performance metrics are summarized in Tables III and IV, using the Hockenheim track as a representative example.

1) Single-Vehicle Setting: Our model demonstrates strong robustness to sensor noise, successfully completing ten laps with up to 30% noise. That is, 108 out of 360 LiDAR beams are set to zero at every timestep, imposing conditions far more demanding than typical real-world sensor failures. Even under 40% noise, the model still completes more than one lap, and under 50% noise, more than half a lap. As noise increases, the model adopts a conservative driving strategy, gradually reducing its speed while maintaining a stable speed variance. This is because our LiDAR spatial pressure token design exerts maximum pressure in the directions affected by sensor noise, prompting the ego vehicle to slow down rather than losing control, thereby preserving smooth and consistent driving behavior even under severe occlusion.

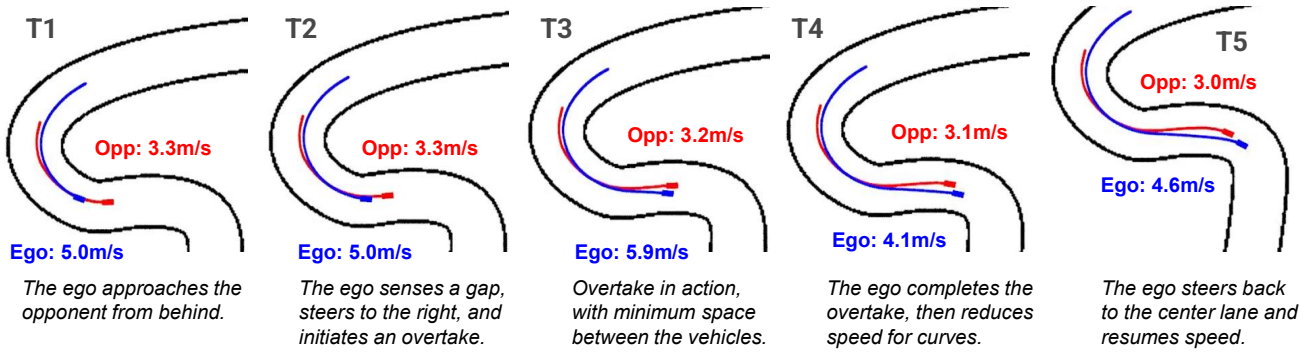

Fig. 4. Overtaking example.

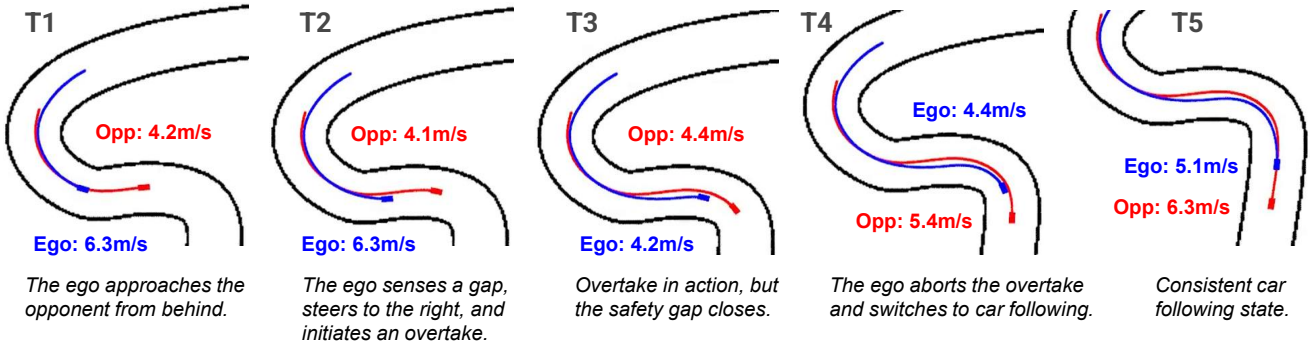

Fig. 5. Car-following example.

TABLE III
SINGLE-VEHICLE PERFORMANCE (HOCKENHEIM) WITH SENSOR NOISE

	Mean Speed (m/s)	Speed Variance (m/s^2)	Mean LapTime (s)	LapTime Variance (s^2)	Laps Completed
10% Noise	5.48	0.16	65.19	0.21	10
20% Noise	4.41	0.18	80.73	0.28	10
30% Noise	3.85	0.21	92.20	0.47	10
40% Noise	2.76	0.10	-	-	1.4
50% Noise	1.97	0.06	-	-	0.6

TABLE IV
HEAD-TO-HEAD RACING PERFORMANCE (HOCKENHEIM) WITH SENSOR NOISE

	Car Following	Overtaking	Collision	Overtake Rate (%)	Safety Rate (%)
10% Noise	421	155	24	25.8	96.0
20% Noise	535	45	20	7.5	96.7
30% Noise	583	2	15	0.4	97.5
40% Noise	579	1	20	0.2	96.7
50% Noise	553	0	47	0.0	92.2

2) Head-to-Head Racing: In head-to-head racing scenarios, our model maintains a high safety rate. As the noise level increases, the model performs fewer overtaking maneuvers and more car-following cases. This indicates that even under heavy sensor noise, the model does not abruptly collide with the opponent. Instead, it slows down and remains cautious until conditions improve.

F. Limitations

Upon reviewing the overtaking cases, we observed that most collisions occur when the ego vehicle returns to the center lane too quickly after completing an overtake, causing its rear to sideswipe the opponent. This behavior does not reflect an inherent limitation of our model, but can be attributed to the imperfect demonstrations of the map-based expert, as discussed in previous sections. While such unsafe cases are filtered during training, our model never explicitly learns how

to avoid them, and is reflected in evaluation. In future work, a more comprehensive expert design that accounts for post-overtake interactions could enable the model to learn safer strategies and further reduce collision rates.

G. Abalation Studies

This section presents ablation studies on key architectural design choices. We evaluate two GRU variants with hidden sizes of $2\times$ (small) and $8\times$ (large) the input dimensionality, as well as a variant that omits the ego-speed input (LiDAR-only). Evaluations are conducted on the Hockenheim track as a representative example.

The small variant achieves an overtaking rate of 55.0%, similar to the baseline, but its safety rate drops to 86.2%, indicating insufficient model capacity to reliably learn car-following and overtaking behavior. The large variant maintains a high safety rate of 92.7% but is overly conservative, with an overtaking rate of only 48.2%. This suggests possible overfitting to training and limited generalization of overtaking skills. The LiDAR-only variant yields higher overtaking rates (60.3%) but at the cost of safety (90.2%). Without explicit ego-speed input, the network issues impulsive commands that deviate from the current state, leading to riskier maneuvers. Since safety is paramount in autonomous racing, we retain ego-speed input to ensure a more balanced policy.

V. Conclusion

In this work, we introduce End2Race, an innovative end-to-end imitation learning algorithm for F1Tenth racing. Our approach achieves high efficiency in single-vehicle settings, adaptive decision-making during overtaking, and strong robustness to sensor noise. Collectively, these results establish it as a competitive autonomous racing solution. Future work will focus on deployment to a physical vehicle and participation in F1Tenth competitions.

References

[1] M. O'Kelly, H. Zheng, D. Karthik, and R. Mangharam, "F1tenth: An open-source evaluation environment for continuous control and reinforcement learning," *Proceedings of Machine Learning Research*, vol. 123, 2020.

[2] Hokuyo Automatic Co., Ltd., *UST-10LX Specification*, 2015.

[3] Intel Corporation, *Intel RealSense D400 Series Product Family Datasheet*, 2022.

[4] M. O'Kelly, V. Sukhil, H. Abbas, J. Harkins, C. Kao, Y. V. Pant, R. Mangharam, D. Agarwal, M. Behl, P. Burgio, and M. Bertogna, "F1/10: An open-source autonomous cyber-physical platform," *arXiv preprint arXiv:1901.08567*, 2019.

[5] S. Macenski and I. Jambrecic, "Slam toolbox: Slam for the dynamic world," *Journal of Open Source Software*, vol. 6, no. 61, p. 2783, 2021.

[6] A. Heilmeier, A. Wischnewski, L. Hermansdorfer, J. Betz, M. Lienkamp, and B. Lohmann, "Minimum curvature trajectory planning and control for an autonomous race car," *Vehicle System Dynamics*, vol. 58, no. 10, pp. 1497–1527, 2020.

[7] C. Walsh and S. Karaman, "Cddt: Fast approximate 2d ray casting for accelerated localization," *arXiv preprint arXiv:1705.01167*, 2017.

[8] M. Pivtoraiko, R. A. Knepper, and A. Kelly, "Differentially constrained mobile robot motion planning in state lattices," *Journal of Field Robotics*, vol. 26, pp. 308 – 333, March 2009.

[9] R. C. Coulter, "Implementation of the pure pursuit path tracking algorithm," Tech. Rep. CMU-RI-TR-92-01, Carnegie Mellon University, Pittsburgh, PA, January 1992.

[10] P. Falcone, F. Borrelli, J. Asgari, H. E. Tseng, and D. Hrovat, "Predictive active steering control for autonomous vehicle systems," *IEEE Transactions on Control Systems Technology*, vol. 15, no. 3, pp. 566–580, 2007.

[11] Z. Zang, H. Zheng, J. Betz, and R. Mangharam, "Local_INN: Implicit map representation and localization with invertible neural networks," in *2023 IEEE International Conference on Robotics and Automation (ICRA)*, pp. 11742–11748, 2023.

[12] F. Muzzini, N. Capodieci, F. Ramanzin, and P. Burgio, "Gpu implementation of the frenet path planner for embedded autonomous systems: A case study in the F1TENTH scenario," *Journal of Systems Architecture*, vol. 154, p. 103239, 2024.

[13] V. Sezer and M. Gokasan, "A novel obstacle avoidance algorithm: "follow the gap method"," *Robotics and Autonomous Systems*, vol. 60, no. 9, pp. 1123–1134, 2012.

[14] T. Hossain, H. Habibullah, R. Islam, and R. V. Padilla, "Local path planning for autonomous mobile robots by integrating modified dynamic-window approach and improved follow the gap method," *Journal of Field Robotics*, vol. 39, no. 4, pp. 371–386, 2022.

[15] X. Sun, M. Zhou, Z. Zhuang, S. Yang, J. Betz, and R. Mangharam, "A benchmark comparison of imitation learning-based control policies for autonomous racing," in *2023 IEEE Intelligent Vehicles Symposium (IV)*, pp. 1–5, 2023.

[16] A. Brunnbauer, L. Berducci, A. Brandstätter, M. Lechner, R. Hasani, D. Rus, and R. Grosu, "Latent imagination facilitates zero-shot transfer in autonomous racing," in *2022 International Conference on Robotics and Automation (ICRA)*, pp. 7513–7520, 2022.

[17] M. M. Zarrar, Q. Weng, B. Yerjan, A. Soyyigit, and H. Yun, "Tinyl-idarnet: 2d lidar-based end-to-end deep learning model for f1tenth autonomous racing," 2024.

[18] J. Zhang and H. Loidl, "F1tenth: An over-taking algorithm using machine learning," in *2023 28th International Conference on Automation and Computing (ICAC)*, pp. 01–06, 2023.

[19] E. Steiner, D. van der Spuy, F. Zhou, A. Pama, M. Liarokapis, and H. Williams, "Accelerating real-world overtaking in f1tenth racing employing reinforcement learning methods," in *2025 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pp. 4528–4534, IEEE, 2025.

[20] K. Cho, B. Van Merriënboer, C. Gulcehre, D. Bahdanau, F. Bougares, H. Schwenk, and Y. Bengio, "Learning phrase representations using rnn encoder-decoder for statistical machine translation," *arXiv preprint arXiv:1406.1078*, 2014.

[21] C. Sammut, "Behavioral cloning," 01 2011.

[22] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, "Proximal policy optimization algorithms," 2017.

[23] S. Ross, G. Gordon, and D. Bagnell, "A reduction of imitation learning and structured prediction to no-regret online learning," in *Proceedings of the fourteenth international conference on artificial intelligence and statistics*, pp. 627–635, JMLR Workshop and Conference Proceedings, 2011.

[24] M. Kelly, C. Sidrane, K. Driggs-Campbell, and M. J. Kochenderfer, "Hg-dagger: Interactive imitation learning with human experts," in *2019 International Conference on Robotics and Automation (ICRA)*, pp. 8077–8083, IEEE, 2019.

[25] J. Spencer, S. Choudhury, M. Barnes, M. Schmittle, M. Chiang, P. Ramadge, and S. Srinivasa, "Expert intervention learning: An online framework for robot learning from explicit and implicit human feedback," *Autonomous Robots*, vol. 46, no. 1, pp. 99–113, 2022.

[26] M. Paluch, F. Bolli, X. Deng, A. Rios Navarro, C. Gao, and T. Delbruck, "Fpga hardware neural control of cartpole and f1tenth race car," 2024.

[27] N. Hamilton, P. Musau, D. M. Lopez, and T. T. Johnson, "Zero-shot policy transfer in autonomous racing: Reinforcement learning vs imitation learning," in *2022 IEEE International Conference on Assured Autonomy (ICAA)*, pp. 11–20, 2022.

[28] T. P. Lillicrap, J. J. Hunt, A. Pritzel, N. Heess, T. Erez, Y. Tassa, D. Silver, and D. Wierstra, "Continuous control with deep reinforcement learning," in *International Conference on Learning Representations*, 2016.

[29] T. Haarnoja, A. Zhou, P. Abbeel, and S. Levine, "Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor," in *International conference on machine learning*, pp. 1861–1870, Pmlr, 2018.

[30] S. Fujimoto, H. Hoof, and D. Meger, "Addressing function approxi-

mation error in actor-critic methods," in *International conference on machine learning*, pp. 1587–1596, PMLR, 2018.

[31] D. Hafner, T. Lillicrap, J. Ba, and M. Norouzi, "Dream to control: Learning behaviors by latent imagination," *arXiv preprint arXiv:1912.01603*, 2019.

[32] T. Dwivedi, T. Betz, F. Sauerbeck, P. Manivannan, and M. Lienkamp, "Continuous control of autonomous vehicles using plan-assisted deep reinforcement learning," in *2022 22nd International Conference on Control, Automation and Systems (ICCAS)*, pp. 244–250, 2022.

[33] J. Bhargav, J. Betz, H. Zheng, and R. Mangharam, "Track based offline policy learning for overtaking maneuvers with autonomous racecars," 2021.

[34] R. Trumpp, E. Javanmardi, J. Nakazato, M. Tsukada, and M. Caccamo, "Racemop: Mapless online path planning for multi-agent autonomous racing using residual policy learning," in *2024 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pp. 8449–8456, 2024.

[35] M. Werling, J. Ziegler, S. Kammel, and S. Thrun, "Optimal trajectory generation for dynamic street scenarios in a frenét frame," in *2010 IEEE International Conference on Robotics and Automation*, pp. 987–993, 2010.

[36] H. Zheng, Z. Zhuang, J. Betz, and R. Mangharam, "Game-theoretic objective space planning," 2023.

[37] A. Sakai, D. Ingram, J. Dinius, K. Chawla, A. Raffin, and A. Paques, "PythonRobotics: A python code collection of robotics algorithms," 2018.

[38] N. Baumann, E. Ghignone, C. Hu, B. Hildisch, T. Hämmerle, A. Bettoni, A. Carron, L. Xie, and M. Magno, "Predictive spliner: Data-driven overtaking in autonomous racing using opponent trajectory prediction," *IEEE Robotics and Automation Letters*, vol. 10, no. 2, pp. 1816–1823, 2025.

[39] R. Geirhos, J. Jörn-Henrik, C. Michaelis, R. Zemel, B. Wieland, B. Matthias, and F. A. Wichmann, "Shortcut learning in deep neural networks," *Nature Machine Intelligence*, vol. 2, pp. 665–673, 11 2020.

[40] X. Sun, S. Yang, M. Zhou, K. Liu, and R. Mangharam, "Mega-dagger: Imitation learning with multiple imperfect experts," 2024.

[41] S. D. Pendleton, H. Andersen, X. Du, X. Shen, M. Meghjani, Y. H. Eng, D. Rus, and M. H. Ang, "Perception, planning, control, and coordination for autonomous vehicles," *Machines*, vol. 5, no. 1, 2017.